NAME DINESHKUMAR SUBJECT Computer Networks

STD. BE IT DIV. A. ROLL NO. 24100154 SCHOOL

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1)		LINUX COMMANDS AND WINDOWS	17/7/2026	
2)		LEARNING AND ASSIGNMENT OF IP ADDRESS MANUALLY TO COMPUTERS	23/7/2026	
3)		STUDY OF DIFFERENT TYPES OF NETWORK CABLES AND CRIMPING OF CABLE WITH RJ45 CONNECTOR	23/7/2026	
5)		DEVELOPE A CUSTOMIZED PING COMMANDS TO TEST THE SERVER COMMUNITY	24/7/2026	
4)		IMPLEMENT PACKET SNIFFING USING RAW SOCKETS IN PYTHON	6/8/2026	
9)		DESIGN A SIMPLE TOPOLOGY AND CONFIGURE WITH ONE ROUTER, TWO SWITCHES AND PCS USING CISCO PACKET TRACER	13/8/2026	
10)		CUSTOMIZE SWITCH WITH NETWORK MODULES USING CISCO PACKET TRACER	13/8/2026	
7)		DEVELOP A SIMPLE CALCULATOR USING XMLRPC	20/8/2026	
12)		NMAP TO DISCOVER LIVE HOSTS USING ARPSCAN, ICMPSCAN AND TCP/UDP PINGSCAN IN TRYHACKME PLATFORM	3/9/2026	

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
6.		BUILDING ANONYMOUS FTP SCANNING USING FTPLIB MODULE	10/9/2026	
12.		EXAMINE NETWORK ADDRESS TRANSLATION (NAT) USING CISCO PACKET TRACER	11/9/2026	
13.		DEMONSTRATE NETWORK FORENSICS USING DCAPXRAY TOOL	24/9/2026	
14.		TO CAPTURE, SAVE AND ANALYZE NETWORK TRAFFIC ON TCP/UDP/IP/ HTTP/ARP/DHCP/ICMP/DNS/USING WIRESHARK TOOL	25/9/2026	
15.		ANALYZE THE DIFFERENT TYPES OF SERVERS USING WEBALIZER TOOL	1/10/2026	
8.		DEVELOP A PROGRAM TO IMPLEMENT CHATTING	10/10/2026	

LINUX COMMANDS AND WINDOWS COMMANDS

i) pwd

Description: prints the current working directory.

output:

$pwd

/home/live user.

ii) mkdir

Description: Creates a new directory

output:

$ mkdir fedora

$ ls

Desktop Documents Downloads fedora

iii) cd.

Description: Changes the current direction

output:

$ cd fedora.

$pwd

/home/liveuser/fedora.

iv) touch filename.txt

Description: creates an empty file

output:

$touch example.txt

$ ls.

example.txt.

v) echo "Hi" > filename.txt

Desciption: writes text into a file

output:

$echo "Hi" > example.txt

vi) cat filename.txt

Desciption: Displays the contents of a file

output:

$ cat example.txt

vii) Hi
grep

Desciption: Searches for a specific pattern in a file

output:

$grep "H" example.txt

viii) Hi
wc

Desciption: Displays the number of lines, words and bytes in a file

output:

$wc example.txt

1 1 3 example.txt

ix) wc -l

Desciption: Counts the number of lines in a file

output:
$ wc -l example.txt
1 example.txt

x) wc -w
Description: counts the number of words in a file

output:
$ wc -w example.txt
1 example.txt

xi) wc -c
Description: count the number of bytes in a file

output:
$ wc -c example.txt
3 example.txt

xii) wc -m
Description: counts the number of characters in a file

output:
$ wc -m example.txt
3 example.txt

xiii) wc -L
Description: Displays the length of the longest line in a file

output:
$ wc -L example.txt
2 example.txt

Basic Networking

hostname

Description: Displays the system's hostname

output:

$ hostname

localhost.localdomain

route

xv) route

Description: Displays the routing table

output:

$ route

Kernel IP routing table

xvi) netstat:

Description: shows network connections and routing information

output:

$ netstat

Active Internet connections

xvii) ip address show

Description: Displays IP address and network interface details

output:

$ip address show

1. lo:

2: enp0s3:

xviii) nslookup

Description: Finds the IP address of a domain

output:

$nslookup www.google.com

Name: www.google.com

Address: 142.250.xxx.xxx

xix) ifconfig.

Description: Displays network interface configuration

output:

$ ifconfig

enp0s3

inet 10.0.2.15

xx) ping

Description: Tests connectivity to a host

output:

$ ping: www.google.com

64 bytes from

RESULT:

Thus the linux and windows commands are studied and executed successfully

EXP.NO:2 LEARNING AND ASSIGNMENT OF IP ADDRESS MANUALLY TO COMPUTERS

DATE: 23/7/2024

AIM:

To learn IP addressing and manually assign an IP address to a computer. To verify the assigned IP address using Linux networking commands.

PRINCIPLES:

* An IP address is a unique numerical address assigned to every device connected to a network.
* It enables communication between devices over a network.
* An IPV4 address consists of four octets separated by dots (e.g. 192.168.1.10)
* In Linux, the ip command is used to display and manage IP addresses, network interfaces, routing tables, and neighbouring devices.

COMMANDS:

i) ip address show

Sample output:

2: ens160:

inet 172.16.211.128/24

fe80::20c:29ff:fe75:9927/64

ii) Display a specific network interface

Command:

ip addr show dev ens160

Sample output:

2: ens160:

inet 172.16.211.128/24

iii) Display Routing Table

Command:

ip route show

Sample output:

default via 172.16.211.2 dev ens160

172.16.211.0/24 dev ens160

iv) Display Neighbor (ARP) Table

Command:

ip neigh show

Sample output:

172.16.211.1 REACHABLE

172.16.211.254 STALE.

manual IP configuration

Parameter	value
IP Address	192.168.1.10
subnet mask	255.255.255.0
Default Gateway	192.168.1.1
DNS server	8.8.8.8

RESULT:

The IP address was manually assigned and verified successfully using Linux networking commands

Exp.No: 3
Date: 23/7/2026

STUDY OF DIFFERENT TYPES OF NETWORK CABLES AND CRIMPING OF CABLE WITH RJ45 Connector.

AIM:

To study different types of network cables and understand the crimping process using an RJ45 Connector.

CONCEPT:

NETWORK cables are used to connect computers and other networking devices for data communication. The most commonly used cable is LAN is the Unshielded Twisted pair (UTP) cable. An RJ45 connector is attached to the T568A or T568B wiring standard before crimping and the completed cable is tested to ensure proper connectivity.

Types of Network cables:

Cable type	Description
1) UTP (Unshielded Twisted pair)	Commonly used for Ethernet LAN's and office networks

2. STP (Shielded Twisted pair) - used in area with high electrical or electromagnetic interference.

3. Coaxial cable - used for cable television, broad based internet and CCTV systems.

4. optical Fiber cable - used for high-speed high bandwidth and long distance communication.

CRIMPING PROCEDURE:

1. cut the UTP cable to the required length.
2. strip about 2-3cm of the outer insulation.
3. untwist and straighten the wires.
4. Arrange the wires according to the T568B standard.
5. Train the wire ends evenly.
6. Insert the wires into the RJ45 connector.
7. CRIMP the connector using a crimping tool.
8. Test the cable using a LAN Cable Tester

OBSERVATION:

* The wires were arranged according to the T568B standard
* The RJ45 connector was crimped successfully
* The cable tester indicated proper continuity.
* The cable was ready for network communication.

RESULTS:

The different types of network cables were studied, and the UTP cable was successfully terminated using and RJ45 connector and tested for proper connectivity

EX:NO:4
DATE:
06/08/
2026

IMPLEMENT PACKET SNIFFING USING RAW SOCKETS IN PYTHON.

AIM:

To implement packet sniffing using raw sockets in python and capture network packets passing through the network interface.

OBJECTIVE:

To understand how raw sockets work, and how network packets can be captured and analysis using python programming.

THEORY:

* Packet sniffing is the process of monitoring and capture data packets travelling across a network.

* A packet sniffer intercepts packets and analyzes the's contents.

* Raw sockets provide direct access to lower-level network protocols, allowing applications to capture packets before they are processed by the operating system.

* In python, the socket module can be used to create raw sockets for packet capture.

* This technique is useful for networking monitoring trouble shooting and security analysis.

ALGORITHM:

1. Import the required socket module
2. Create a raw socket.
3. Bind the socket to the network's interface.
4. Receive packets using the recvfrom()
5. Display the captured packet Information.
6. Repeat the process continuously to monitor network traffic.

PROGRAM:

```
import socket
sniffer = socket.socket(socket.
    AF_INET, SOCKET, SOCK_RAW,
    SOCKET.IPPROTO_IP)
host = "192.168.1.5"
```

```
sniffer.bind((host,0))
while True:
    raw_data,addr=sniffer.recvfrom(65535)
    print("packet received from:"addr)
    print(raw_data)
```

OUTPUT:

packet received from: ('192.168.1.5',0)

b'E\x00\x00\x00...'

packet received from: ('192.168.1.10',0)

b'E\x00\x00\x00...'

RESULT:

Thus, packet sniffing was successfully implement using raw sockets in python and network packets were captured and displayed.

EX.NO:5

DATE:
24/7/2026.

DEVELOP A CUSTOMIZED PING COMMANDS TO TEST THE SERVER CONNECTIVITY

AIM:

To develop a customized ping command using python of the windows command prompt (CMD) to test server connectivity, measure response time, identify packet loss and verify network communication between the local system and a remote server.

THEORY:

The ping command is a network diagonistic utility available in the windows command prompt. It is used to determine whether a remote server or computer is reachable over a network. If the network destination responds, the network connection is active, other wise the server may be unreachable or blocked. The purposes are

* TO check server connectivity
* TO measure network responsetime

COMMANDS:

1. Check Basic server connectivity

Command:

Ping google.com

output:

Pinging google.com [142.250.183.78] with 32 bytes of data:

Reply from 142.250.183.78:
bytes=32 time=20ms TTL=118

Reply from 142.250.183.78:
bytes=32 time=21ms TTL=118

Reply from 142.250.183.78:
bytes=32 time=22ms TTL=118

Reply from 142.250.183.78:
bytes=32 time=21ms TTL=118

Ping statistics for 142.250.183.78:
Packets sent = 4, received = 4, lost = 0 (0% loss)

2. Send a specific number of ping requests

Command:

Ping google.com -n 5

Sample output:

Packets: sent = 5, Received = 5, Lost = 0

Approximate round trip times:
Minimum = 20ms
Maximum = 22ms
Average = 21ms

3. Continuous ping:

Command:

ping google.com -t

Sample output:

Reply from 142.250.183.78:
bytes =32 time =20ms TTL = 118

Reply from 142.250.183.78
bytes =32 time =21ms TTL=118

4. Display Help for Ping Command:

Command:

ping /?

Display all available options and syntax for the ping command in window command prompt

output:

Usage: ping [-t] [-a] [-n count] [-l size] [-f]
[-i TTL] [-v TOS] [-r count]
[-s count] [-j host-list] |
[-k host-list]] [-w timeout]
destination

RESULT:

The customized ping commands were successfully executed in the command Prompt

EX: NO:4
DATE: 13/8/2024

DESIGN A SIMPLE TOPOLOGY AND CONFIGURE WITH ONE ROUTER, TWO SWITCHES AND PCS USING CISCO PACKET TRACKER.

AIM:

To design and configure a simple network topology consisting of one router, two switches, and multiple PCs in Cisco packet tracer, and enable communication between PC's.

DEFINITIONS:

* Router: A networking device that forwards data packets between computer networks, directing traffic on the internet.

* Switch: A device that connects devices within a network and uses MAC addresses to forward data to the correct destination.

* PC: End devices in a network that users interact with to send and receive data.

* Straight-through cable: used to connect different devices like PC to switch or switch to router,

* Crossover cable: Used to connect similiar devices, like switch to switch.

* Console Cable: Used to connect PC to router or switch console port for configuration.

EQUIPMENT NEEDED

* CISCO Packet Tracer Software.
* 1 Router (e.g. 2811)
* 2 Switches (e.g., 2960)
* 4 PCs
* Appropriate cables (Copper Straight-Through)

STEP-BY-STEP PROCEDURE:

Step 1: Open Cisco Packet Tracer
Launch Cisco Packet Tracer on your computer.

Step 2: Add Devices to workspace
From the button device toolbar, drag and drop.

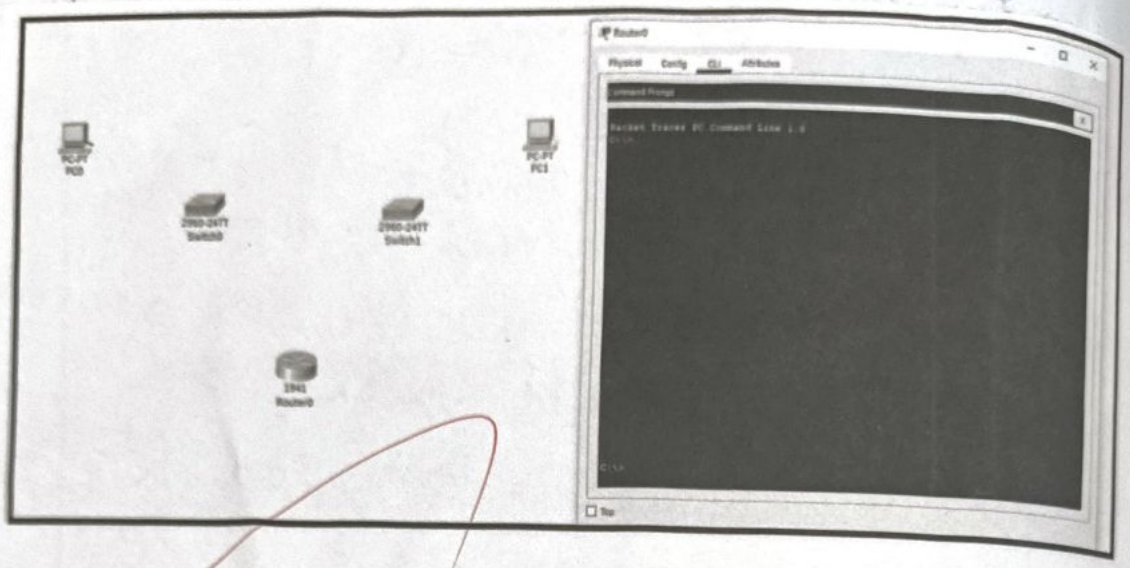
- 1 router (1941)
- 2 switches (2960)

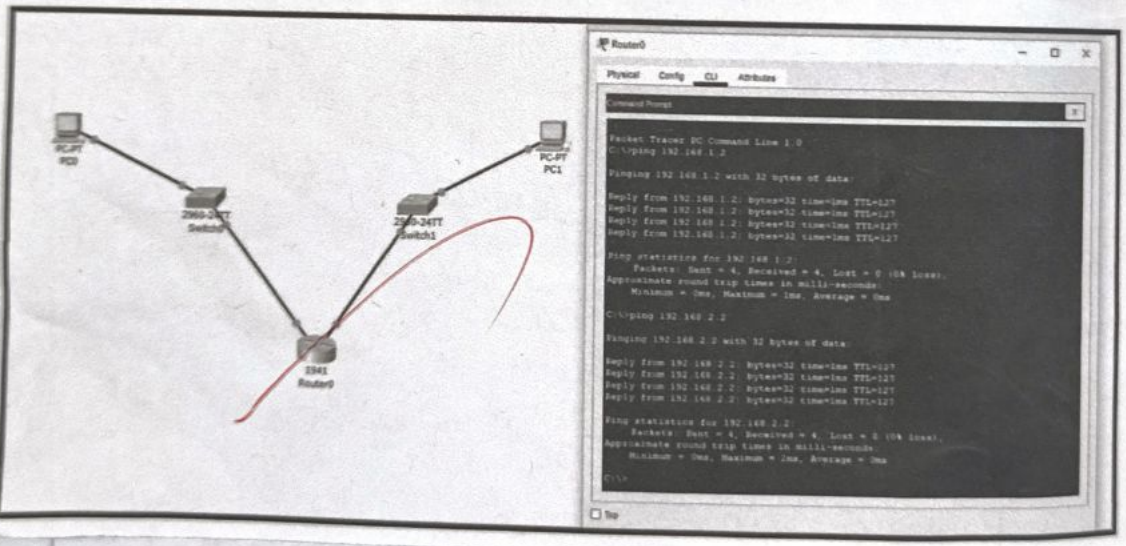
• 4 PCs (2 connected to each switch)

Step 3: Connect Devices with cables
• use copper straight-Through cables to connect.
• Router Gigabit Ethernet 0/0 to Switch 0 FastEthernet 0/1
• Router GigabitEthernet 0/1 to Switch 1 FastEthernet 0/1
• PCs to to the switches.
• PC0 to switch 0 FastEthernet 0/2.
• PC1 to switch0 FastEthernet 0/3.
• PC2 to switch 1 FastEthernet 0/2
• PC3 to switch 1 Fast ethernet 0/3.

Step 4: Configure Router Interfaces
click the router > CLI tab. Enter the following commands: enable
configure terminal interface
gigabit Ethernet 0/0 ip address
192.168.1.1 255.255.255.0 no shutdown
exit interface.

Step 5: Configure PC IP Addresses
• click on each PC > Desktop tab > IP configuration and assign the following IPs and gateways:





RESULT:

The PC's connected to different switches can communicate with each other via the router. The successful Ping replies verify that the network is properly configured.

EX:NO:10 CUSTOMIZE SWITCH WITH NETWORK
20/8/2024 MODULES USING CISCO PACKET TRACER.

AIM:

To learn how to add a network module to a cisco switch, connect multiple PC's and a server, assign IP addresses and test connectivity in cisco packet tracer.

DESCRIPTION:

This is a network module is an extra component you can add to a switch. It gives you more ports or adds new types of conversations. In packet tracer, modules such as HWIC-4ESW let you expand the switch's capabilities

PROCEDURE:

1. Open cisco packet tracer and set up your workspace.
2. Drag and drop a 2960-24TT switch to the center.
3. Add five PCs and one server -PT- from the End Devices Panel.
4. Power off the switch

5. Install HWIC-4ESW module on the switch.

6. Power on the switch and let it boot.

7. Connect all devices using copper straight-Through cables:

- PC0 -> switch FastEthernet 0/1
- PC1 -> switch FastEthernet 0/2
- PC2 -> switch module port 1
- PC3 -> switch module port 2
- PC4 -> switch module port 3

8. Test connectivity

Input:

=

Devices:

- 1 Cisco 2960-24TT switch
- 1 HWIC-4ESW network module
- 5PCs (PC0-PC4)
- 1 server-PT (Served)

Cables:

=

- copper straigh through cables

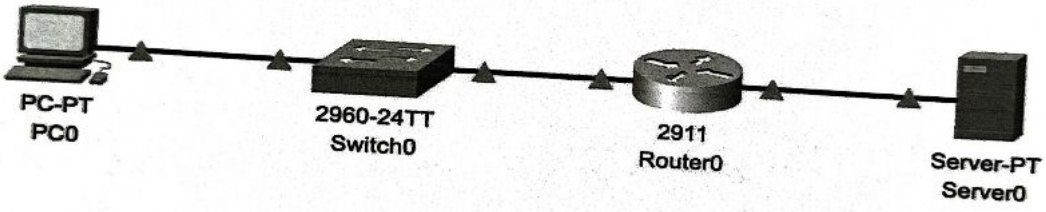
IP configuration

- Use static IP addresses as listed above

OUTPUT:

=

A working network setup where every PC and the server can successfully ping each other, proving that both built-in and module ports on the switch are functioning.



RESULT:

The experiment demonstrates successful customization of a switch with a network module and connectivity between all devices. The network works as expected, and all devices communicate

EX:NO:7 DEVELOP A SIMPLE CALCULATOR USING XML RPC

28/8/2026

AIM:

To develop a simple calculator application using the XML-RPC protocol in Python that performs basic arithmetic operations like addition, subtraction, multiplication, and division through client server communication.

DESCRIPTION:

XML-RPC is a protocol that uses XML to encode requests and responses and HTTP as a transport mechanism. It enables communication betweens a client and a server where the client invokes methods hosted on the server remotely.

- The server hosts arithmetic functions.
- The client connects to the server to perform operations like

addition, subtraction multiplication and division.

CODE:

1. Server-side code - server.py

```
from xmlrpc.server import
SimpleXMLRPCServer

def add(x,y):
    return x+y
def subtract (x,y):
    return x-y
def multiply (x,y):
def divide (x,y):
    if y!=0:
        return x/y
    else:
        return "Error : Division by zero"
server = SimpleXMLRPCServer(("
    localhost",8000), logrequest
    =False)
server.register_function(add,"add")
server.register_function(subtract,"subtract")
server.register_function(multiply,"multiply")
server.register_function(divide,"divide")
print("XML-RPC calculator server is
    running on port 8000...")
```

server.serve_forever()

2. client-side code - client.py

```
import xmlrpc.client
proxy = xmlrpc.client.ServerProxy
("http://localhost:8000/")
print("--- XML-RPC Calculator Test
---")
num1, num2 = 30, 10
print("Inputs: 30 and 10")
print("Addition (+) : ",
      proxy.add(num1, num2))
print("Subtraction (-) : ",
proxy.subtract(num1, num2))
print("multiplication (*) : ",
proxy.multiply(num1, num2))
print("Division (/)  : ",
proxy.divide(num1, num2))
```

output:

--- XML-RPC Calculator Test ---
Inputs: 30 and 10
Addition (+) : 40
Subtraction (-) : 20
Multiplication (*) : 300

```
from xmlrpc.server import SimpleXMLRPCServer
from xmlrpc.client import ServerProxy
import threading


def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Cannot divide by zero"
    return a / b
```

```
===== XML-RPC CALCULATOR =====
Enter first number: 2
Enter second number: 5

Choose operation:
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice: 4

Result: 0.4
```

PROCEDURE:

1. Open a Python IDE or terminal
2. Create two separate Python files: server.py and client.py.
3. Write the provided server-side code in server.py and client-side code in client.py.
4. Run the server.py file first to start the XML-RPC server.
5. Run the client.py file to perform arithmetic operations remotely
6. Observe the results displayed on the client screen and verify the server output.

RESULT:

The XML-RPC-based calculator was successfully implemented.

EX:NO:12 NMAP TO DISCOVER LIVE HOSTS
3/9/2026 USING ARP SCAN, ICMP SCAN,
AND TCP/UDP PING SCAN IN
TRYHACKME PLATFORM.

AIM:

To learn and implement Nmap host-discovery techniques (ARP, ICMP variants, TCP, UDP pings) to identify live hosts on a network and compare their effectiveness.

DESCRIPTION:

Use Nmap probe types to discover which IPs are active before port/vulnerability scans. Covers ARP (LAN), ICMP Echo/Timestamp/mask, TCP SYN/ACK, UDP probes, and combined scans with optional packet tracing or disabling ARP.

PROCEDURE:

ARP (LAN) nmap -sn -PR 10.0.X.0/24
nmap -sn -PR 192.168.1.0/24 --packet-trace
ICMP Echo/Timestamp/Mask nmap -sn -PE 10.10.X.0/24 nmap -sn -PP 10.10.X.100 nmap -sn -PM 10.10.X.100

#TCP SYN/ACK (specify ports)
nmap -sn -PS22,80,443 10.10.X.0/24
nmap -sn -PA80,443 10.10.X.0/24

#UDP nmap -sn -PU53,161 10.10.X.0/24
combined/disable ARP nmap -sn -PE
-PS -PA -PU 10.10.X.0/24 nmap -sn
10.10.X.0/24 --disable-arp-ping INPUT
Target IP or range (eg., 10.10.X.100 or
10.10.X.0/24)
optional port lists for -PS, -PA-PU
(eg., 22,80,443,53,161)

Flags: -sn, -PR-PE, -PP, -PM, -PS, -PA, -PU,--
packet-trace, --disable-arp-ping.

Output:
ARP
Nmap scan report for 10.10X.1 Host
is up (0.00050s latency).
MAC Address: 02:83:75:3A:F2:89
Nmap done: 256 IP addresses (3 host up)
ICMP
Nmap scan report for 10.10.X.100 Host
is UP (0.00099s latency).
Nmap done: 1 IP address (1 host up)

RESULT:
ARP discovery reliably identified
hosts on the local subnet with
the lowest latency and highest
accuracy.

EX.NO:6 BUILDING ANONYMOUS FTP SCANNER
10/9/2024 USING FTPLIB MODULE

AIM:

To create a python-based FTP-scanner using the ftplib module that.

* To check if a public FTP server allows anonymous login.
* Lists the contents of directories on the server.

PROCEDURE:

* Import the built in ftplib module
* write two python scripts:
* sender side: Connects to the FTP server, logs in anonymously and lists the root directory
* Receiver side: Connects anonymously and lists a specific sub directory.
* Run the scripts on the google colab or any python environment.

code:

Sender side code:

```
ftplib
def anonymousLogin(hostname):
    try:
        ftp=ftplib.FTP(hostname)
            =ftp.login('anonymous',
                       'anonymous')
        anonymous login
        print(response)
        if "230" in response:
            +str(hostname)+" FTP
                Anonymous Login.
            print("welcome message:\n", ftp.
                getwelcome())
            print("\n Directory listing:")
            ftp.dir() ftp.quit()
        else:
            print("FTP Anonymous login Failed")
anonymousLogin(hostname)
```

Receiver side code:

```
Import ftplib
def check_anonymous_connection(hostpath):
    try:
        with ftplib.FTP(host) as connection:
            connection.login('anonymous',
                             'anonymous')
```

```
print ("welcome:", connection.getwelcome())
print(f"\nListing directory: {path}:")
connection.cwd(path)
for name, details in connection.mlsd():
    print(name, details['type'],
          details.get('size', 'N/A'))
except Exception as e:
    print("Error:", str(e))

check_anonymous_connection(FTP_SERVER,
                           CRL,
                           DOWNLOAD_DIR_PATH)
```

OUTPUT:

Sender side output:

(230 Anonymous access granted, restrictions apply [+]

ftp.be.debian.org FTP Anonymous Login Succeeded.

welcome message:

220 PROFTPD server (mirror.as35701.net) Directory

Listing:

drwxr-xr-x 9 ftp ftp

Receiver side output:

welcome: 220 PROFTPD server (mirror.as35701.net)

Listing directory : /pub/linux/kernel/v5.x/

linux-5.10.1.tar.xz file 113072564

Linux-5.10.2.tar.xz file 113167444

RESULT:

The script successfully connected to the FTP server.

EX: NO: 11 EXAMINE NETWORK ADDRESS TRANSLATION
11/9/2026 (NAT) USING CISCO PACKET TRACER

AIM:

Implement network address Translation (NAT) using cisco packet Tracer. We'll configure static, dynamic and port Address Translation (PAT) on a router to enable communication between a private internal network and a public external network.

PROCEDURE:

step 1: setup the Network Topology. Launch the Cisco Packet Tracer software.

step 2: Add Devices.

step 3: Connect Devices.

step 4: Verify connections.

step 5: Assign IP addresses.

step 6: Verify IP configuration.

CODE:

#Enable configure terminal

interface GigabitEthernet 0/0 i/p address 192.168.1.1. 255.255.255.0

no shutdown

exit.

interface GigabitEthernet 0/1 i/p address 200.1.1.1 255.255.255.0
no shutdown
exit

#4.1 Configure Inside and outside Interfaces for NAT

configure terminal interface GigabitEthernet 0/0 ip nat inside exit interface GigabitEthernet 0/1 ip nat outside exit

#4.2 static NAT Configuration (Example)

inside source static 192.168.1.10 200.1.1.10
ip nat pool MY-POOL 200.1.1.20 200.1.1.25 netmask 255.255.255.0
access-list 1 permit 192.168.1.0 0.0.0.255
ip nat inside source list 1 pool MY-POOL

TO SET DEFAULT GATEWAY:

1. Click on PC 1 in the workspace.
2. Go to the Desktop tab.
3. Click on IP Configuration.
4. In the Default Gateway field, type: 192.168.1.1.
5. Make sure the IP address and subnet mask are already set
6. Close the window.

RESULT:

The router successfully translated private IPS to public one using static, dynamic and PAT configurations.

EX: NO: 13
24/9/2026

DEMONSTRATE NETWORK FORENSICS USING PCAPXRAY TOOL

AIM:

To analyze and visualize network traffic from a PCAP file using the PCAPXRay tool for forensic investigation.

STEPS TO COMPLETE NETWORK FORENSICS:

Experiment using pcapxray

step 1: prepare your computer

* make sure you have python 3 installed

step 2: Install Pcapxray Tool

* open command prompt or Terminal
* Type this command and press Enter.

* pip install pcapxray

This installs the pcapxray tool on your computer.

step 3: Get a PCAP File

* Download a sample PCAP file from a website like https://wiki.wireshark.org/SampleCaptures.
* save the file on your computer.
* open wireshark
* click on the network interface to start capturing.

Step 4: Run Pcap Xray on your PCAP File

* put the PCAP file in a folder you can easily access.
* cd path-to-your-folder
* Run this command
* pcapxray -f capture.pcap

Step 5: View the Results

* After the command runs, it will create a file named pcapxray.html
* open this file in your web browser
* you will see a visual map of network traffic showing:

Step 6: Analyze what you see

* Look for the biggest nodes - these are the most active devices.
* Check if any strange IP addresses are communicating.
* Notice unusual ports or repeated connections

RESULT:

PCAPXray successfully generated a visual map of hosts, connections and protocols, helping identify suspicious or abnormal network activities.

EX.NO:4
25/9/2024

TO CAPTURE, SAVE, AND ANALYZE NETWORK TRAFFIC ON TCP/UDP/IP/HTTP/ARP/DHCP/ICMP/DNS USING WIRESHARK TOOL

AIM:

The aim of this experiment is to capture, save, and analyze network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP/DNS using wireshark Tool.

PROCEDURE:

Step 1: Download and install wireshark from its official website. Ensure you have administrator privileges for network capture.

step 2: Launch wireshark and identify the network interface through which your device is connected to the network

Step 3: Before starting the capture, apply capture filtees to focus on specific protocols and reduce unwanted data. Example of capture filtees. you can use capture only TCP traffic: tcp.

Step 4: Click the start button to begin capturing live network traffic on the chosen interface.

Step 5: While capturing, use display filters to isolate and analyze specific protocols more easily. Examples of display only TCP packets: tcp

To display only UDP packets: udp

To display only HTTP packets: http

Step 6: Let the capture run until you have collected enough packets for analysis, then click the stop button to halt the capture process.

Step 7: Save the captured traffic by navigating to file.

Step 8: Review and analyze the captured packets in detail. Look for key information such as source and destination IP addresses, protocol types, packet lengths and flags.

Output:

No: packet number in the capture sequence

Time: Timestamp when the packet was captured.

Source: The source IP address of the packet

Destination: The destination IP address of the packet

protocol: The protocal used (e.g. TCP, UDP, HTTP, ARP, DNS)

Length: The size of the packet in byte

Sample output:

No	Time	source	Destination	protocal	Length
1	0.00000	192.168.1.1	8.8.8.8	ICMP	56
2	0.00000	8.8.8.8	192.168.1.2	ICMP	64
3	0.2945	192.168.1.2	192.168.1.2	ARP	42

RESULT:

Thus this experiment capture, save and analyze network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP/DNS using wireshark tool.

EX.NO: 15
1/10/2026

ANALYZE THE DIFFERENT TYPES OF SERVERS USING WEBALIZER TOOL

AIM:

To analyze the usage statistics of different types of servers (Apache, Nginx, FTP/proxy) using the webalizer tool.

PROCEDURE:

1. Install the webalizer tool in the system:

sudo apt-get update sudo apt-get install webalizer

2. Verify Installation:

webalizer -V

3. Locate the log files of the servers:

Apache: /var/log/apache2/access.log

Nginx: /var/log/nginx/access.log

FTP/proxy: /var/log/vsftpd.log or /var/log/apache

4. Run webalizer on the log file to generate a report:

5. Open a web browser and access the report:

6. Repeat the same process for Nginx, FTP or proxy logs.

7. Compare and analyze the reports

PROGRAM (COMMAND):

Step 1: Install webalizer sudo apt-get install webalizer -y

step2: Run webalizer for Apache logs sudo webalizer /var/log/apache2/access.log -o var/www/html/apache_report/

Step3: Run webalizer for Nginx logs

sudo webalizer /var/log/nginx/access.log -o /var/www/html/nginx_report/

step 4: Run webalizer for proxy logs sudo webalizer /var/log/squid/access.log -o /var/www/html/proxy_report/

output:

Sample observation Table

Server
Type
Total
Hits

Bandwidth used

TOP

Client / Domain Report URL MIA

Apache 12,560 1,240 1.8GB 192.168.1.10

Nginx 9,840980 1.2GB 192.168.1.20

Proxy 15,300 1,450 2.1GB 192.168.1.30

Graphical output (as seen in browser):

monthly summary chart

Daily/hourly usage histogram

pie chart of top hosts and URLs

RESULT:

The webalizer tool successfully generated graphical and statistical reports of different servers.

EX.NO:8
10/10/2026

DEVELOP A PROGRAM TO IMPLEMENT CHATTING

AIM:

To develop a simple chatting application using python socket programming to exchange messages between a client and a server.

PROCEDURE:

1. Install python and open VS code.
2. Create two files : server.py and client.py.
3. Import the socket and threading modules.
4. Create a TCP socket in both programs.
5. Bind and start the server then connect the client.
6. Run the server first and the client next.
7. Exchange messages between the two terminals.

8. Type exit to end the chat.

PROGRAM:

Server.py

```
import socket
import threading,
server = socket.socket()
server.bind(("127.0.0.1", 12345))
server.listen()
print("waiting for client....")
conn, addr = server.accept()
print("client connected")
def receive():
    while True:
        msg = conn.recv(1024).decode()
        if not msg or msg.lower() == 'exit':
            break
        print("client:", msg)

while True:
    msg = input("server:")
    conn.sendall(msg.encode())
    if msg.lower() == "exit":
        break
conn.close()
server.close()
```

client.py

```
import socket
import threading

client = socket.socket()
client.connect(("127.0.0.1", 12345))
print("connected to server")

def receive():
    while True:
        msg = client.recv(1024).decode()
        if not msg or msg.lower() == "exit":
            break
        print("client:", msg)

threading.Thread(target=receive, daemon=True).start()

while True:
    msg = input("server: ")
    conn.sendall(msg.encode())
    if msg.lower() == "exit":
        break
```

Sample output:

Server:

Waiting for client....

Client connected

Server: Hello

Client: Hi

Server: How are you?

Client: I am fine

Client

Connected to server

Client: Hi

Server: Hello

Client: I am fine

Server: How are you?

RESULT:

Thus a simple chatting application was implemented successfully using Python socket programming.